STUDENT MANAGEMENT SYSTEM

PROGRAMMING WITH PYTHON

(202044504)

A Mini Project Report submitted in partial fulfilment of the requirements for the Degree of

Bachelor of Technology in Information Technology

Semester V

SUBMITTED BY

Daksh Bhavsar

(12402080601022)

Batch: 1A8

ACADEMIC YEAR 2026-27

A D Patel Institute of Technology, New Vallabh Vidyanagar
Vitthal Udyognagar - 388121, Anand (Gujarat), India

CERTIFICATE

This is to certify that **Mr. Daksh Bhavsar**, a student of the **Bachelor of Technology (Information Technology)** programme, Semester V, has prepared the mini project entitled **"Student Management System"** for the course **"Programming with Python"** (Course Code: 202044504).

This project demonstrates the practical application of Python programming concepts, Object-Oriented Programming (OOP), dictionaries, Tkinter graphical user interface (GUI), and file-based data storage using JSON. The project report is submitted in partial fulfilment of the academic requirements of the Professional Core Course.

We appreciate the efforts made by the student in preparing this mini project.

Date: _____

Place: _____

Course Faculty
Mr. Nayan Mali
Signature: _____

1. Abstract

The Student Management System is a desktop application developed using Python and Tkinter. It helps an institute maintain student records, including student ID, name, course, semester, email and contact details. Users can add new students, view records, search for a student, update information and delete records through a graphical interface. Student data is stored locally in a JSON file, so a separate MySQL server is not required. The project demonstrates Object-Oriented Programming, dictionaries, GUI programming, file handling, input validation and exception handling.

2. Introduction

Educational institutions need to maintain accurate information about students. Managing records manually in notebooks or separate documents can take time and may lead to duplicate entries or difficulty finding information. This project provides a simple computer-based system to organize student details in one place. The graphical interface makes common operations accessible to a user with basic computer knowledge. It is designed as an educational prototype and can be extended with attendance, marks, reports and user authentication.

3. Problem Statement

Manual student-record management can make searching, updating and maintaining information difficult. Records may become inconsistent, duplicated or misplaced, and repeated manual entry can cause errors. The project addresses these problems by providing a digital student table, search functionality, validated data entry and operations to add, update and delete records. Data is saved to a local JSON file to remain available after the application closes.

4. Objectives

- Maintain student details in a structured format.
- Add new student records and display them in a table.
- Search for a student using an ID or name.
- Update existing student information and remove unwanted records.
- Validate required fields and avoid duplicate student IDs.
- Save and load records using JSON file handling without MySQL.
- Demonstrate OOP, dictionaries, Tkinter and exception handling.

5. Scope

The application is intended for a small class, a department demonstration or classroom use. It supports local record management on one computer and uses a JSON file as its storage mechanism. It does not require an internet connection or database server. The current prototype does not provide multiple-user access, cloud synchronization, role-based permissions, automated backups or integration with an institution's official student information system.

6. Hardware and Software Requirements

- Hardware: desktop or laptop, keyboard, mouse and sufficient memory to run Python.
- Operating system: Windows, macOS or Linux with a desktop environment.
- Language: Python 3.
- GUI library: Tkinter and ttk.
- Data storage: local JSON file (for example, students.json).
- Editor: Visual Studio Code, IDLE or any Python IDE.

7. System Design

The application is organized into three logical layers. The presentation layer contains the Tkinter window, student form, search field, table and action buttons. The application layer validates user input and performs add, search, update and delete operations. The storage layer reads student records from and writes records to a local JSON file. Separating interface operations from storage functions makes the program easier to understand, test and maintain.

8. Main Modules

- Student entry module: accepts student ID, name, course, semester, email and contact details.
- Display module: shows stored student records in a tabular Treeview.
- Search module: filters or locates records by student ID or name.
- Update module: loads a selected record into the form and saves revised details.
- Delete module: removes the selected student after confirmation.
- Storage module: loads JSON data at startup and saves changes to the file.
- Validation module: checks required fields and prevents duplicate student IDs.

9. Data Design

The project stores data in a local file named **students.json**. Each student is represented as a record containing the following fields:

Field	Purpose	Example / Rule
student_id	Unique identifier for a student	Required; must not be duplicated
name	Student's full name	Required text
course	Programme or course name	Required text
semester	Current semester	Valid semester value
email	Email address	Optional or validated email
contact	Phone/contact number	Text to preserve leading zeroes

The JSON file stores a list of student records. The student ID acts as the logical unique key checked by the application; there is no SQL database or database server.

10. Object-Oriented Programming

The project can use a StudentManagementApp class to group the graphical interface and student-management operations. Methods can handle adding, selecting, searching, updating and deleting student records. Separate helper functions or a storage class can load and save JSON data. Objects keep related behaviour together and make the program more maintainable than placing all operations in a single long function.

11. Use of Dictionaries and JSON

A Python dictionary stores the fields belonging to one student, such as student ID, name, course, semester, email and contact. A list of dictionaries represents all student records. Dictionaries make it straightforward to access and update a particular field by its key. The JSON module converts the list of dictionaries into a text file and reads it back when the program starts, providing persistence without MySQL.

12. User Interface

The main window contains input fields for student details, a search field, a table of existing students and buttons for Add, Update, Delete, Search, Clear and Refresh. When the user selects a row, its values are copied into the form for editing. The table is refreshed after a successful operation so that the displayed information matches the saved data. Message boxes communicate validation errors and confirm successful operations.

13. Algorithm

- Start the program and create the Tkinter main window.
- Load student records from students.json; use an empty list if the file does not yet exist.
- Display the records in the student table.
- Accept input and validate required fields and student ID uniqueness.
- For Add, append a new student record; for Update, modify the selected record.
- For Delete, ask for confirmation and remove the selected record.
- For Search, match the entered student ID or name and display the result.
- Save changed records to JSON and refresh the table.
- Display a message and continue until the user closes the application.

14. Testing

Test case	Expected result	Status to verify
Launch application	Window opens and saved records load	Run locally
Add valid student	Record appears in table and JSON file	Run locally
Add duplicate ID	Duplicate record is rejected	Run locally
Search by ID/name	Matching student is displayed	Run locally
Select a table row	Selected student's details fill the form	Run locally
Update student	Edited values are saved and shown	Run locally
Delete student	Confirmed record is removed	Run locally
Restart application	Previously saved records are restored	Run locally
Submit blank required fields	Validation warning appears	Run locally

Testing should be performed on the student's own computer. The status column is intentionally a verification checklist rather than a claim that all tests have already been executed.

15. Exception Handling and Data Safety

The application should handle missing required fields, duplicate student IDs, invalid semester values, file-not-found conditions and JSON decoding or file-writing errors with clear messages. Before updating or deleting a record, the application should ensure a student has been selected. Deletion should require confirmation. The JSON file should be backed up regularly, and sensitive student information should not be committed to a public repository.

16. Limitations and Future Enhancements

- Add administrator login and role-based access.
- Add attendance, marks and examination-result modules.
- Generate student lists and academic reports in PDF or CSV format.
- Add advanced filters for course, semester and student status.
- Provide automatic backup and restore options.
- Move from local JSON storage to a database when the project grows.
- Add a web or mobile interface and multi-user support.

17. Conclusion

The Student Management System provides a practical example of applying Python to an everyday academic task. It combines a Tkinter graphical interface, dictionary-based records, Object-Oriented Programming and JSON file handling. The application makes it easier to add, find, update and delete student information while keeping records available between sessions. By avoiding MySQL, the project remains simple to set up and suitable for a beginner-level mini project. It also provides a foundation for future enhancements such as attendance tracking, marks management, reporting and secure multi-user access.

18. References

- Python documentation: https://docs.python.org/3/
- Tkinter documentation: https://docs.python.org/3/library/tkinter.html
- Python JSON module documentation: https://docs.python.org/3/library/json.html
- Python tutorial: https://docs.python.org/3/tutorial/